

CS & IT ENGINEERING



Operating System

Process Synchronization

Lecture -2

By- Vishvadeep Gothi sir



Recap of Previous Lecture



Doubt ?

Topic

Synchronization

Topic

Race Condition



Topics to be Covered



Topic

Critical Section

Topic

Mutual Exclusion

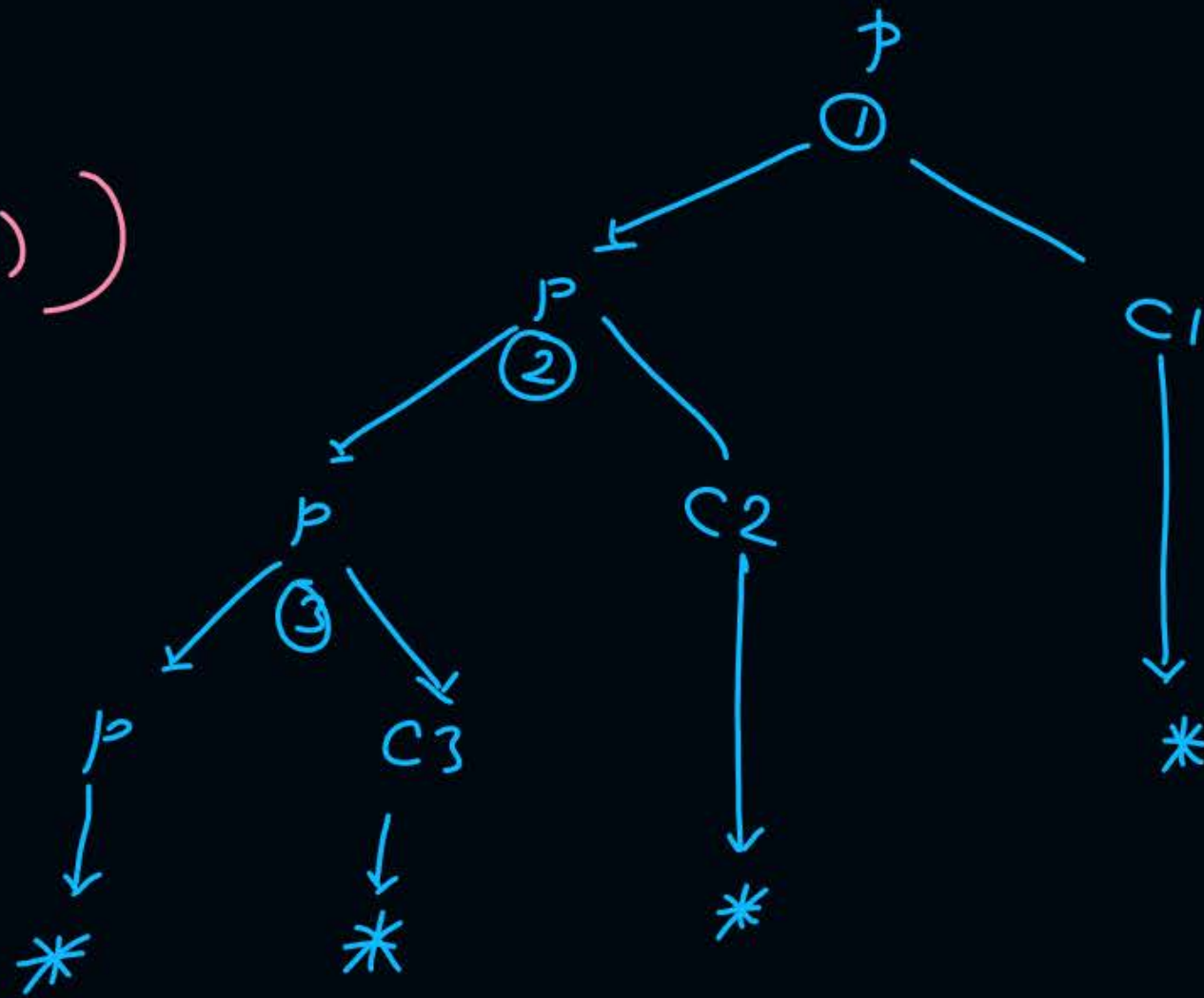


Interprocess communication $\Rightarrow$

1. shared variable or resource or memory
2. message passing
3. pipe

```
if(fork1() && fork2())  
{  
    fork3();  
    printf("*");  
}
```

Ans = 4



#Q. Consider the following code snippet using the `fork()` and `wait()` system calls. Assume that the code compiles and runs correctly, and that the system calls run successfully without any errors.

```
int x = 3;  
while(x > 0) {  
    fork();  
    printf("hello");  
    wait(NULL);  
    x--;  
}
```

The total number of times the `printf` statement is executed is 14?

GATE-2024

[NAT]

Consider three concurrent processes P1, P2 and P3 as shown below, which access a shared variable D that has been initialized to 100.

P1	P2	P3
⋮	⋮	⋮
$D = D + 20$	$D = D - 50$	$D = D + 10$
⋮	⋮	⋮

The processes are executed on a uniprocessor system running a time-shared operating system. If the minimum and maximum possible values of D after the three processes have completed execution are X and Y respectively, then the value of $Y - X$ is 80.

$$100 + 20 - 50 + 10$$

$$\begin{array}{r}
 Y = 130 \\
 X = 50 \\
 \hline
 Y - X = 80
 \end{array}$$

Ques) In prev. questⁿ all possible distinct values of D 7?

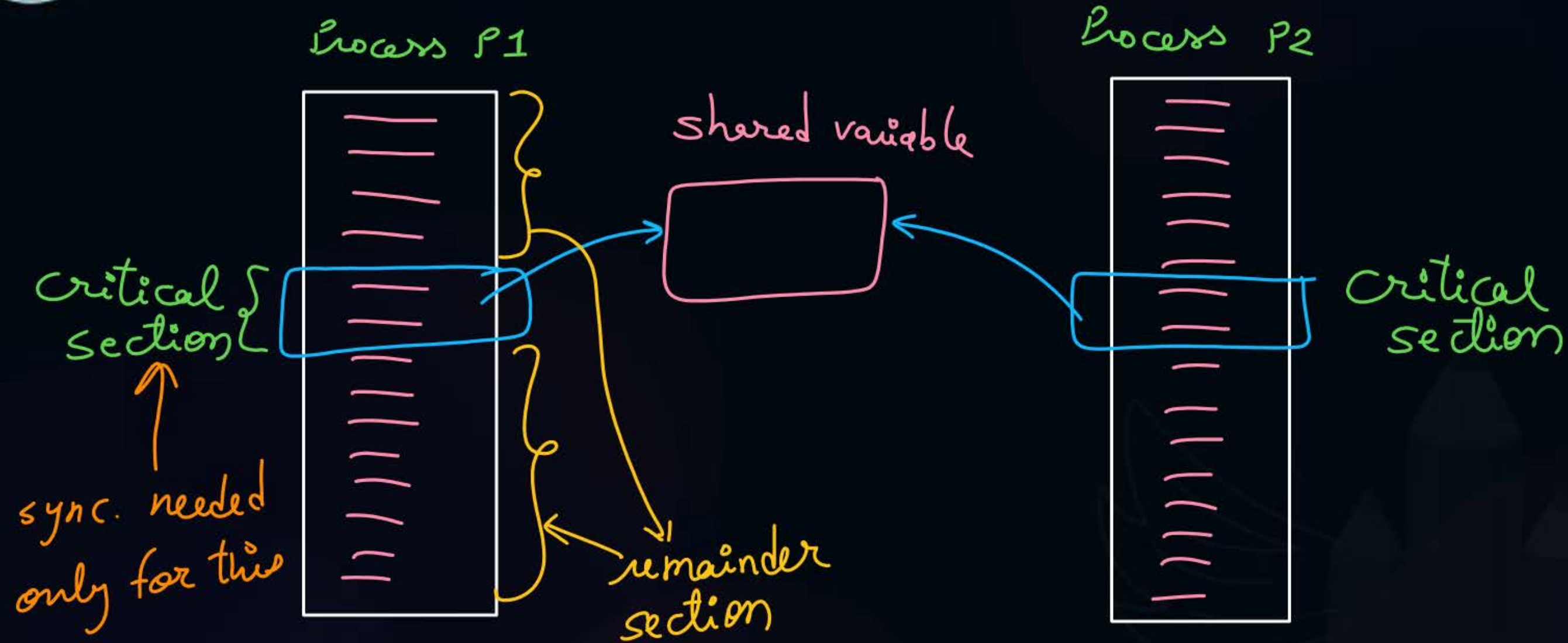
Solⁿ

$$100 + 20 - 50 + 10$$

120 , 70 , 80
110 , 130 ,
50 , 60 ,



Topic : Entire Process Requires Synchronization?





Topic : Critical Section



The critical section is a code segment where the shared variables can be accessed.

↓
communication happens with other process.

Problem of critical section :-

OS should provide synchronizatⁿ for critical sections in such a way that the final result should be the desired result.



Topic : Solution of Critical Section Problem

Requirements of Critical Section problem solution:

1. Mutual Exclusion
2. Progress
3. Bounded Waiting

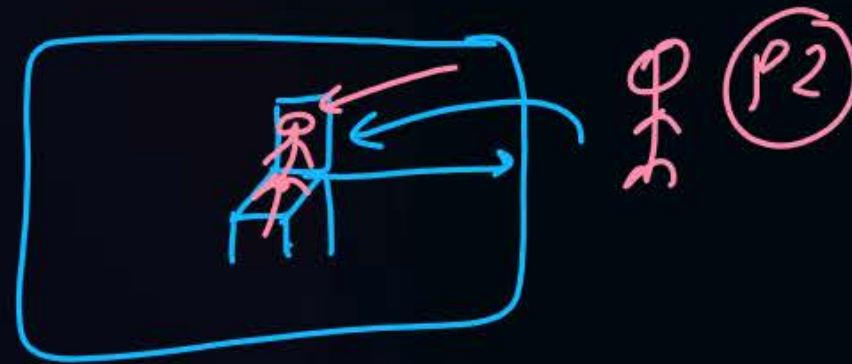
} Perfect solution should satisfy all 3 requirements



Topic : Solution of Critical Section Problem

Mutual Exclusion:

If one process is executing the critical section, then other process is not allowed to enter into critical section.

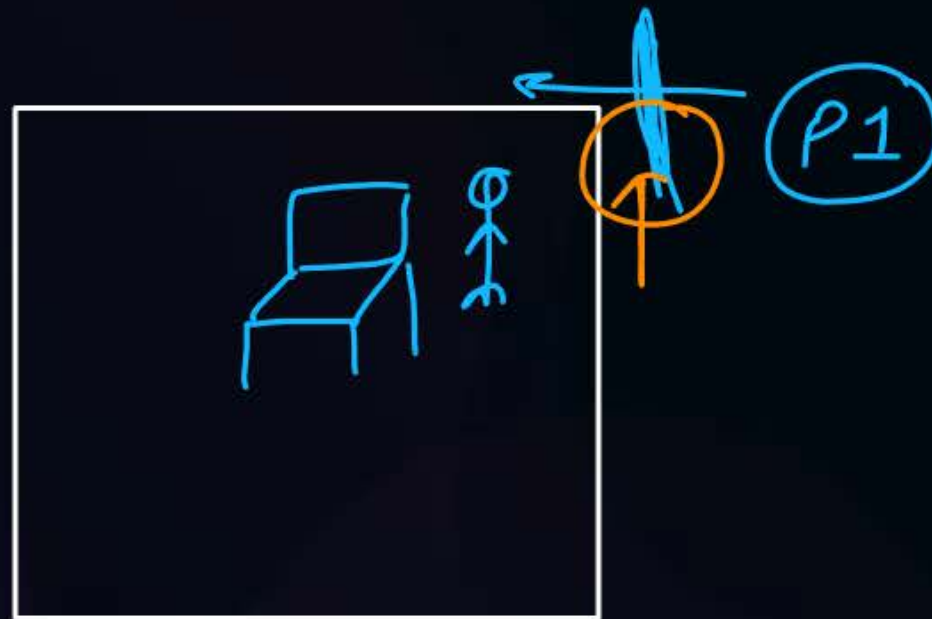




Topic : Solution of Critical Section Problem

Progress:

If no any process is in critical section and any process wants to enter into critical section, then the process must be allowed.

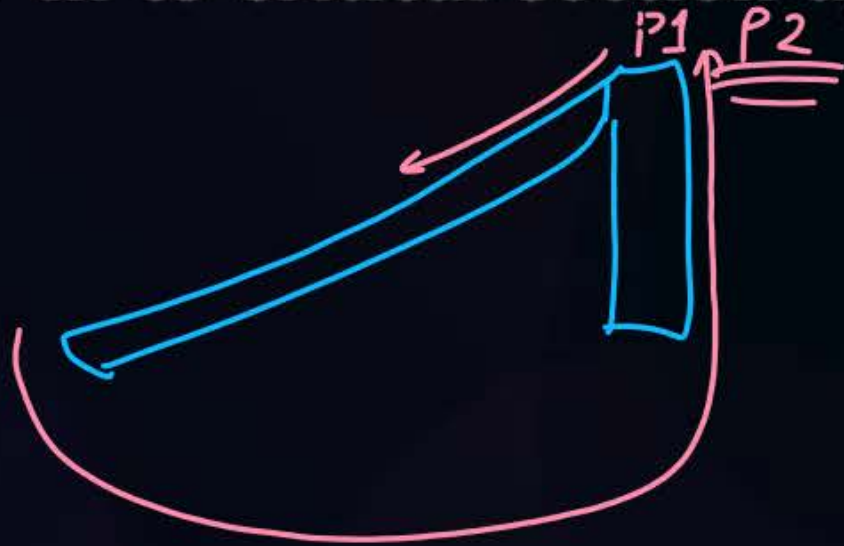




Topic : Solution of Critical Section Problem

Bounded Waiting:

If a process p_1 is executing in critical section and other process p_2 is waiting for critical section, then the waiting time of p_2 must be bounded. Which means p_1 must not enter in to critical section again and again by keeping p_2 in waiting for long.





Topic : 2-Process Solution



Process



Entry section

critical section (C.S.)

exit section

← ensures all 3 conditions to satisfy

← for announcement that the process left C.S.

Solution 1

shared variable

Boolean lock=false;

c.s. not locked
true ⇒ c.s. locked

P0

```
while(true)
{
    while(lock);
    lock=true;
    //CS
    lock=false;
    RS;
}
```

P1

```
while(true)
{
    while(lock);
    lock=true;
    //CS
    lock=false;
    RS;
}
```

✗ mutual Exclusion

✓ progress

✗ Bounded waiting

⇓

one process in c.s.
other process waiting
for c.s.



2 mins Summary

Topic

Critical Section

Topic

Mutual Exclusion



Happy Learning

THANK - YOU

